

Chapter 1

Foundations of Convolutional Neural Networks

Computer vision has undergone a big paradigm shift over the past two decades. Historically, automated visual recognition systems relied heavily on hand-crafted feature extractors—such as Scale-Invariant Feature Transform (SIFT), and Histogram of Oriented Gradients (HOG)—by human domain experts to capture specific geometric, spatial, or textural invariants. While these engineered representations succeeded in constrained settings, they routinely fractured under the weight of real-world complexity: drastic illumination changes, non-rigid object deformations, occlusion, and background clutter.

The resurgence of deep artificial neural networks, spearheaded by parallel GPU computing architectures and the availability of massive labeled datasets such as ImageNet, fundamentally redefined the field. Rather than engineering fixed, hand-crafted descriptors, modern computer vision leverages end-to-end differentiable learning. Convolutional Neural Networks (CNNs) systematically extract hierarchical feature representations directly from raw pixel arrays, transforming low-level spatial primitives into rich, high-level semantic representations of visual scenes.

Today, computer vision powers foundational technologies across modern society, from real-time perception in autonomous vehicles and medical diagnostic imaging to multi-modal generative models. This introductory chapter lays the bedrock for the rest of this book, establishing the fundamental concepts, pivotal network architectures, practical engineering routines, and transfer learning workflows required to master modern visual deep learning.

1.1 Computer Vision Concepts

1.1.1 Review of Convolutional Networks

Convolutional layers are particularly well suited to images because they introduce three important structural properties:

- **Local connectivity:** Each output activation depends only on a small spatial neighborhood of the input, rather than on every input pixel. This exploits the fact that nearby pixels tend to contain strongly related visual information.
- **Parameter sharing:** The same kernel weights are applied at every spatial location. Consequently, a filter can detect the same pattern wherever it appears in the image while using far fewer parameters than a fully connected layer.
- **Translation Equivariance:** Shifting the input image shifts the output feature map by the exact same amount. In other words, moving an object does not change *what* features are detected, only *where* they appear. For a unit-stride convolution (ignoring boundary effects), this is expressed as:

$$\text{Conv}(T_{\Delta}X) = T_{\Delta} \text{Conv}(X),$$

where T_{Δ} denotes a spatial shift by Δ .

Together, local connectivity and parameter sharing allow convolutional layers to learn efficient spatial feature detectors, while translation equivariance allows those detectors to respond consistently across the image. With these properties in mind, we now define the convolutional operation mathematically.

As a preliminary reference for the upcoming sections, we recall the **standard mathematical operation** defining a two-dimensional spatial convolution layer. Given an input feature tensor $X \in \mathbb{R}^{C_{\text{in}} \times H \times W}$ zero-padded by p pixels on each side to yield a padded tensor $\tilde{X} \in \mathbb{R}^{C_{\text{in}} \times (H+2p) \times (W+2p)}$, and a learnable kernel filter $K \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k \times k}$, the activation map $Y \in \mathbb{R}^{C_{\text{out}} \times H' \times W'}$ at channel index c and spatial coordinate (i, j) is expressed as:

$$Y_{c,i,j} = b_c + \sum_{l=1}^{C_{\text{in}}} \sum_{m=1}^k \sum_{n=1}^k \tilde{X}_{l,i \cdot s + m \cdot d, j \cdot s + n \cdot d} K_{c,l,m,n}$$

where b_c is the bias parameter for output channel c , s denotes the spatial stride governing the step size of the filter traversal, d is the dilation rate defining the spacing between kernel elements, and k represents the spatial kernel size. The effective kernel size expanded by dilation is $k_{\text{eff}} = d(k - 1) + 1$. Consequently, the output spatial dimensions (H', W') are calculated via:

$$H' = \left\lfloor \frac{H + 2p - d(k - 1) - 1}{s} \right\rfloor + 1, \quad W' = \left\lfloor \frac{W + 2p - d(k - 1) - 1}{s} \right\rfloor + 1$$

An example is shown in Figure 1.1, where the dilation is 1.

Note

Strictly speaking, this expression is cross-correlation because the kernel is not spatially flipped. Deep-learning literature and libraries conventionally call it convolution, and the kernel is learned, so the distinction does not reduce the layer's expressive power.

Understanding how zero-padding, stride, and dilation control boundary information, spatial downsampling, and receptive field expansion, respectively, is critical as we build and analyze modern convolutional architectures throughout this chapter.

Example: 2D Convolution

1. Dimensional Setup

- Input (X): Shape $(C_{\text{in}}, H, W) = (2, 3, 3)$
- Kernel (K): Shape $(C_{\text{out}}, C_{\text{in}}, k, k) = (3, 2, 2, 2)$
- Bias (b): Shape $(C_{\text{out}}) = (3)$
- Hyperparameters: Stride $s = 1$, Padding $p = 0$, Dilation $d = 1$
- Output (Y): Shape $(C_{\text{out}}, H', W') = (3, 2, 2)$

2. Input Tensors and Parameters

Input Channels (X):

$$X_1 = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \\ 2 & 1 & 0 \end{bmatrix}, \quad X_2 = \begin{bmatrix} 0 & 2 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Learnable Filters and Biases:

- Filter Set 1 (for Output Channel 1, with $b_1 = 1$):

$$K_{1,1} = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}, \quad K_{1,2} = \begin{bmatrix} 0 & 2 \\ 1 & 0 \end{bmatrix}$$

- Filter Set 2 (for Output Channel 2, with $b_2 = -1$):

$$K_{2,1} = \begin{bmatrix} 2 & 1 \\ 0 & -1 \end{bmatrix}, \quad K_{2,2} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

- Filter Set 3 (for Output Channel 3, with $b_3 = 0$):

$$K_{3,1} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad K_{3,2} = \begin{bmatrix} -1 & 0 \\ 0 & 2 \end{bmatrix}$$

3. Resulting Output Tensor (Y)

The resulting feature maps for each output channel are:

$$Y_1 = \begin{bmatrix} 8 & 3 \\ 0 & 4 \end{bmatrix}, \quad Y_2 = \begin{bmatrix} 0 & 3 \\ 1 & 4 \end{bmatrix}, \quad Y_3 = \begin{bmatrix} 0 & 3 \\ 4 & 6 \end{bmatrix}$$

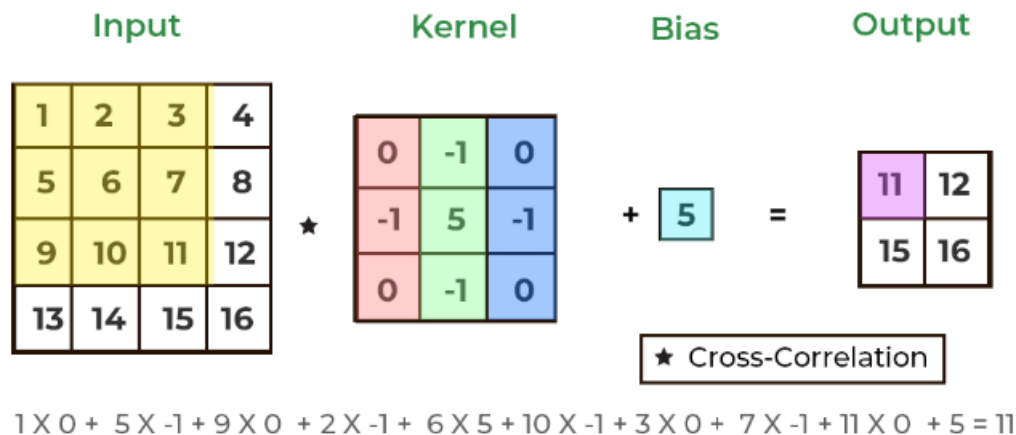


Figure 1.1: Example of a convolutional operation.

1.1.2 Features along the Network

A convolutional neural network (CNN) processes an image hierarchically, extracting increasingly abstract representations as we move through its layers. This architecture partially mimics the functioning of the human brain’s visual cortex, where the first neurons detect simple edges and subsequent layers respond to more complex patterns.

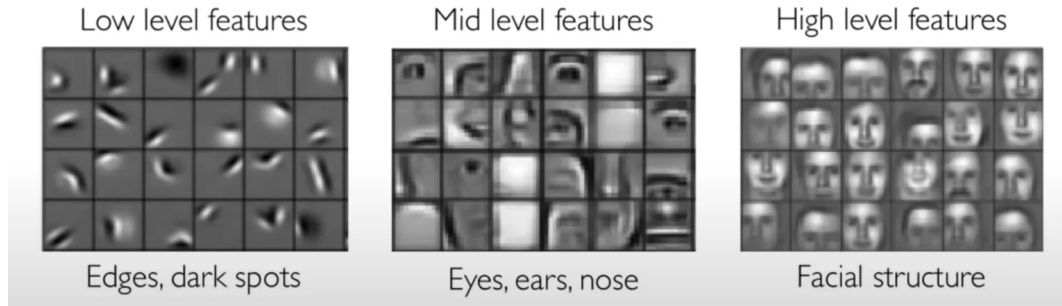


Figure 1.2: Different layers capture different features.

In the **first layers**, convolutions learn very **local and simple features**: basic edges, corners, textures, and colors. These filters act as generic shape detectors, responding to patterns such as horizontal, vertical, or diagonal lines. Spatial information is largely preserved, and the features have a clear visual meaning.

In **intermediate layers**, the features are combined and begin to represent **more complex parts**, such as curves, specific textures, or combinations of edges that can correspond to parts of objects (eyes, wheels, leaves, etc.). At this point, the activated patterns become more dependent on the semantic content of the image.

Finally, in the **deep layers**, activations correspond to **high-level, highly abstract features**, which can be specialized to detect entire objects or even specific classes. These representations are no longer necessarily easily visualized or directly interpretable by humans,

but they encapsulate the relevant information the model needs to classify or recognize what is in the image.

This progression, from simple to complex, is key to understanding both the power and difficulty of explaining CNNs. While the first features can be easily interpreted visually, those in the deep layers require more sophisticated visualization and attribution techniques to understand their function.

1.1.3 Receptive Field

Mathematical Concept of Receptive Field Expansion

In this section, we assume a unit stride ($s_i = 1$) and no dilation ($d_i = 1$) for every convolutional layer i .

The **receptive field** defines the spatial region in the original input image (measured as the number of input pixels) that contributes to the activation of a single unit in a deeper feature map. Under our assumptions ($s = 1, d = 1$), the receptive field grows linearly and purely **additively** layer by layer, meaning a 2D image region would span $RF \times RF$ input pixels).

By recursively applying this additive property across L stacked layers, the **general equation** of the receptive field simplifies to

$$RF_L = RF_{L-1} + (k_L - 1) = 1 + \sum_{i=1}^L (k_i - 1),$$

where k_i is the kernel size for layer i and $RF_0 = 1$.

If all layers use an identical kernel size k , the receptive field simplifies further to $RF_L = 1 + L(k - 1)$.

Note

The **theoretical receptive field** contains every input location that can influence the layer activation. In a trained network, these locations generally contribute with different strengths. The region exerting the strongest influence is called the **effective receptive field** and may be smaller than the theoretical one.

Example: Receptive field of a CNN with two layers

Let Layer 1 denote the original input image, and let RF_l denote the side length of the theoretical receptive field of an activation in Layer l , measured in Layer 1 pixels. The kernel size is $k = 3$ for all layers. A single input pixel has

$$RF_1 = 1.$$

For stride $s = 1$ and dilation $d = 1$, each convolutional layer increases the receptive-field side length by $k - 1$:

$$RF_l = RF_{l-1} + (k - 1).$$

Consider two consecutive 3×3 convolutional layers. Applying the recurrence gives

$$RF_2 = RF_1 + (3 - 1) = 3,$$

$$RF_3 = RF_2 + (3 - 1) = 5.$$

Therefore, an activation in Layer 3 has a theoretical receptive field of 5×5 pixels in the original input and an activation in Layer 2 has a theoretical receptive field of 3×3 pixels in the original input.

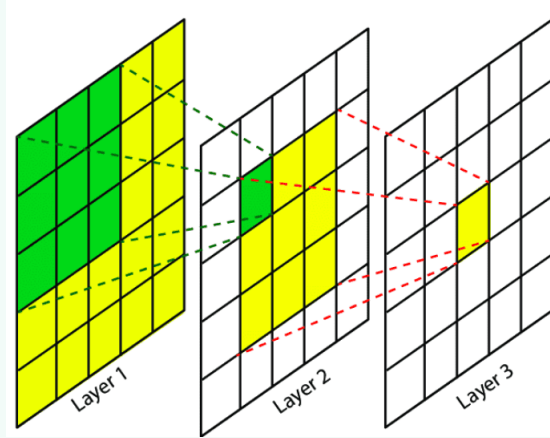


Figure 1.3: Backward receptive-field trace from one Layer 3 activation (yellow) and from one Layer 2 activation (green).

Let's focus on the backward trace of Layer 3 in Figure 1.3. A single activation in Layer 3 depends on a 3×3 region of Layer 2. At the same time, a single activation in Layer 2 depends on a 3×3 region of Layer 1. Because the receptive fields of adjacent Layer 2 units overlap, their union covers a 5×5 region of Layer 1.

Feature Hierarchy: From Local Specifics to Global Abstractions

This step-by-step additive expansion of the receptive field directly drives the hierarchy of feature representations learned by the network (note the relation with the previous section):

- **Shallow Layers (Small Receptive Field):**

- *Spatial Scope:* Small, highly localized visual context (e.g., 3×3 or 5×5 pixels).
- *Feature Type:* **Specific, low-level visual features.**
- *Characteristics:* Because these units only observe tiny patches of the original input, they learn simple local primitives such as oriented edges, color transitions, corners, and fine textures.

- **Intermediate Layers (Moderate Receptive Field):**

- *Spatial Scope:* Regional context incorporating multiple local patterns (e.g., 9×9 or 13×13 pixels).
- *Feature Type:* **Mid-level structural features.**
- *Characteristics:* By combining primitive edges and textures from shallow layers, these units learn geometric shapes, repeated patterns, curves, and specific object parts (e.g., eyes, wheels, or corners).

- **Deep Layers (Large Receptive Field):**

- *Spatial Scope:* Broad context covering a substantial portion (or the entirety) of the input space.
- *Feature Type:* **General, high-level semantic abstractions.**
- *Characteristics:* Units in deep layers aggregate information from hundreds of overlapping local feature detectors. As a result, they trade fine-grained spatial position detail for robust, invariant semantic representations of complete objects and complex visual concepts (e.g., recognizing a face, vehicle, or scene type).

1.1.4 Data Augmentation

Conceptual Rationale and Intuition

Modern deep neural networks have millions of parameters, making them prone to simply memorizing exact training images rather than learning true visual concepts. If a model only sees fixed, static images during training, it can quickly overfit to specific pixel arrangements and fail to generalize to new data.

Data augmentation tries to solve this by leveraging a simple intuition: *an image subjected to realistic variations still represents the same object*. For instance, a picture of a cat tilted slightly to the left, cropped, or under different lighting is still fundamentally a cat (see Figure 1.4).

By applying different random transformations during every training epoch, data augmentation effectively provides a virtually endless stream of new training samples. This prevents the network from memorizing static pixel patterns, forcing it to learn robust, generalizable features instead.

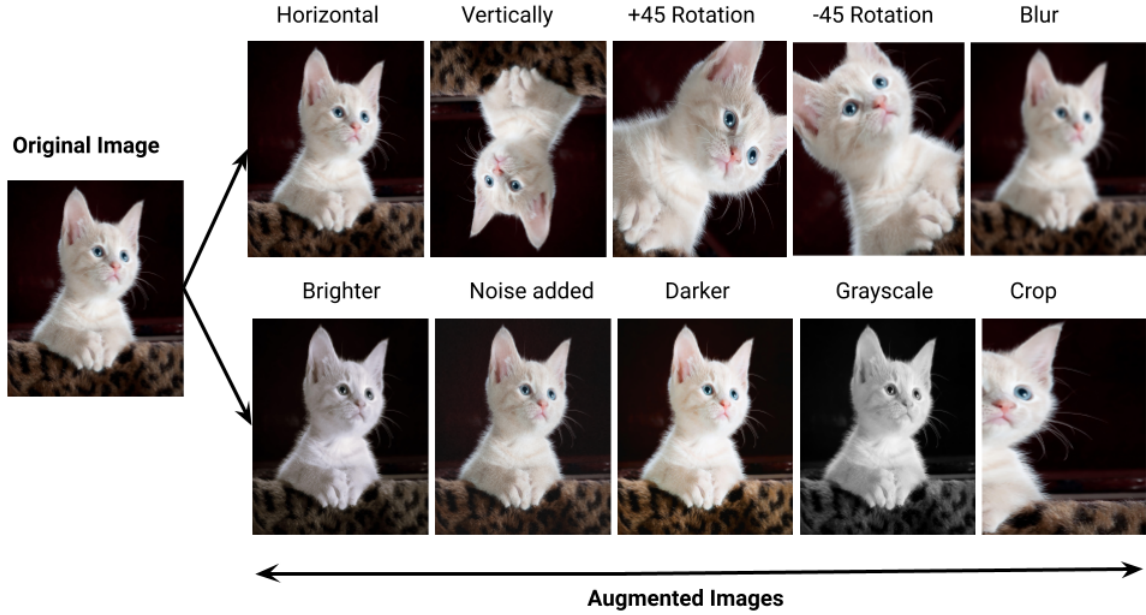


Figure 1.4: Data augmentation for the image of a cat.

Geometric Transformations

Geometric operations alter pixel spatial coordinates, encouraging the network to learn spatial invariants such as translation, scale, and orientation invariance.

- **Random Cropping and Scaling:** Extracts a random spatial subset of the image tensor and resizes it to a target dimension (H, W) .
- **Flips and Rotations:** Applies horizontal/vertical reflections or discrete rotations around the image center.
- **Affine Transformations:** Applies geometric mappings using a matrix $M \in \mathbb{R}^{3 \times 3}$:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Photometric Transformations

Photometric operations modify intensity values without changing spatial coordinates, conferring robustness against illumination and sensor variations.

- **Color Jittering:** Perturbs brightness, contrast, saturation, and hue components.
- **Noise and Smoothing:** Injects Gaussian noise $\varepsilon \sim \mathcal{N}(0, \sigma^2)$ or applies Gaussian blur kernels to reduce dependence on high-frequency noise.

Multi-Sample Mixing and Information Dropping

Advanced techniques enforce global contextual awareness by obscuring local visual cues or interpolating between multiple distinct samples:

- **Cutout / Random Erasing:** Sets a random rectangular patch of the input image to zero, preventing the model from relying on a single dominant feature.
- **Mixup:** Forms convex combinations of pairs of inputs (x_i, x_j) and their targets (y_i, y_j) governed by $\lambda \sim \text{Beta}(\alpha, \alpha)$:

$$\tilde{x} = \lambda x_i + (1 - \lambda)x_j, \quad \tilde{y} = \lambda y_i + (1 - \lambda)y_j$$

- **CutMix:** Overlays a spatial cutout patch from x_j onto x_i , scaling ground-truth label proportions to the relative binary mask area M :

$$\tilde{x} = M \odot x_i + (1 - M) \odot x_j, \quad \tilde{y} = \lambda y_i + (1 - \lambda)y_j$$

We will see more about this technique in Chapter 6.

Impact on Model Representations

By restricting memorization, data augmentation directly shapes the feature spaces in deep networks. Invariant transformations force deep feature maps to encourage reduced sensitivity (such as minor lighting changes), whereas multi-sample interpolation smooths class decision boundaries, reducing output overconfidence on out-of-distribution inputs.

1.2 Computer Vision Architectures

1.2.1 Standard CNN Baseline

Before the advent of modern deep architectures, a typical Convolutional Neural Network (CNN) followed a standardized, two-part pipeline. The first stage acts as a local feature extractor, consisting of alternating convolutional layers (to detect spatial patterns) and pooling layers (to downsample spatial dimensions and provide translation invariance). The second stage acts as a global classifier; the final spatial feature maps are flattened into a 1D vector and passed through a Multi-Layer Perceptron (MLP) consisting of Fully Connected (FC) layers. This is represented in Figure 1.5. While conceptually simple, this approach suffers from parameter explosion and struggles to optimize effectively as depth increases. Both AlexNet and VGGNet (architectures we will see in this section) follow this pattern.

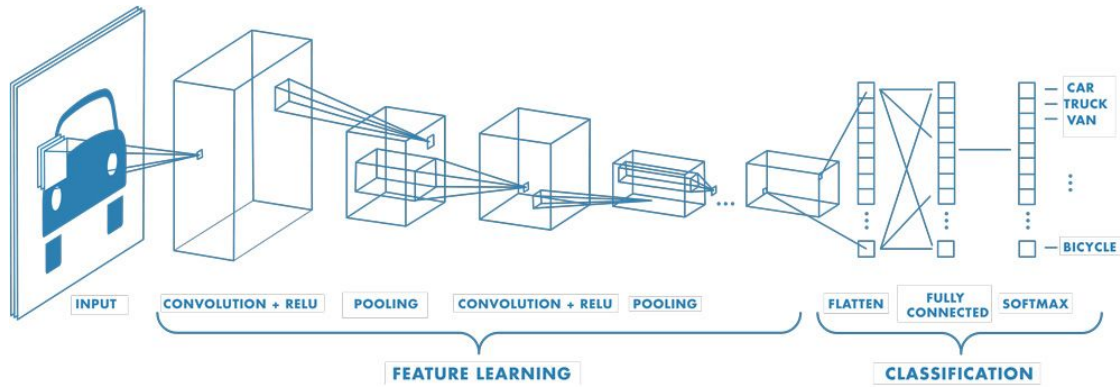


Figure 1.5: Standard CNN baseline architecture.

1.2.2 AlexNet (2012)

AlexNet is a landmark deep convolutional neural network that catalyzed the deep learning revolution by winning the 2012 ImageNet competition. It consists of 5 convolutional layers followed by 3 fully connected layers. A diagram is shown in Figure 1.8.

Key innovations:

- **ReLU Activations:** Replaced traditional sigmoid/tanh functions with the Rectified Linear Unit (ReLU), which drastically accelerated convergence and mitigated the vanishing gradient problem.
- **Dropout Regularization:** It helped popularize stochastic neuron dropout in the dense layers to reduce overfitting.
- **GPU Utilization:** Pioneered the use of parallelized GPU training to scale network capacity.
- **Data Augmentation:** Extensively utilized label-preserving transformations (scaling, cropping, flipping) to artificially expand the dataset and improve generalization.

1.2.3 VGGNet (2014)

VGGNet formalized the concept of architectural homogeneity, showing empirically that network depth is a critical component for high representational capacity. An schema is shown in Figure 1.9.

Key innovations:

- **Micro-Architecture Design:** Exclusively utilized highly localized 3×3 convolution filters with a stride of 1.
- **Receptive Field Equivalence:** Demonstrated that stacking two 3×3 convolutions yields an effective receptive field of 5×5 , while requiring fewer parameters and introducing more non-linearities into the mapping.

- **Deep Homogeneity:** Scaled up to 16 and 19 weight layers (VGG-16, VGG-19) using a highly uniform, repeating block structure (Convolutions $\rightarrow$ ReLU $\rightarrow$ Max Pooling), which simplified network design and transfer learning.

1.2.4 GoogLeNet / Inception v1 (2014)

GoogLeNet (named as a nod to LeNet and the “we need to go deeper” meme from the movie *Inception*) abandoned the sequential, stack-based design of its predecessors in favor of a wider, parallelized topology, as shown in Figure 1.6. The full architecture is shown in Figure 1.7.

Key innovations:

- **Inception Modules:** Processed the input at multiple scales simultaneously by executing parallel 1×1 , 3×3 , and 5×5 convolutions—alongside max pooling—and concatenating their feature maps.
- **Bottleneck Layers:** Heavily employed 1×1 convolutions to compress channel depth before expensive 3×3 and 5×5 operations, drastically reducing the computational budget.
- **Global Average Pooling (GAP):** Substantially reduced parameter-heavy fully connected layers at the network’s tail with GAP, collapsing each spatial feature map into a single scalar, effectively dropping millions of parameters and preventing overfitting.
- **Auxiliary Classifiers:** To avoid the vanishing gradient problem, GoogleNet uses auxiliary classifiers, which are mini-classification branches attached to the middle of the network. Instead of making the early layers wait for a diluted signal from the very end, these intermediate branches calculate their own error and inject fresh, strong gradients directly into the middle of the architecture. During training, the network’s total error is a combination of the main output’s loss and a weighted fraction (usually 30%) of the auxiliary classifiers’ losses, ensuring that learning remains active across all layers. Because these branches are strictly training aids designed to stabilize the learning process, they are completely discarded during inference, meaning the network relies solely on its main final output when making actual predictions.

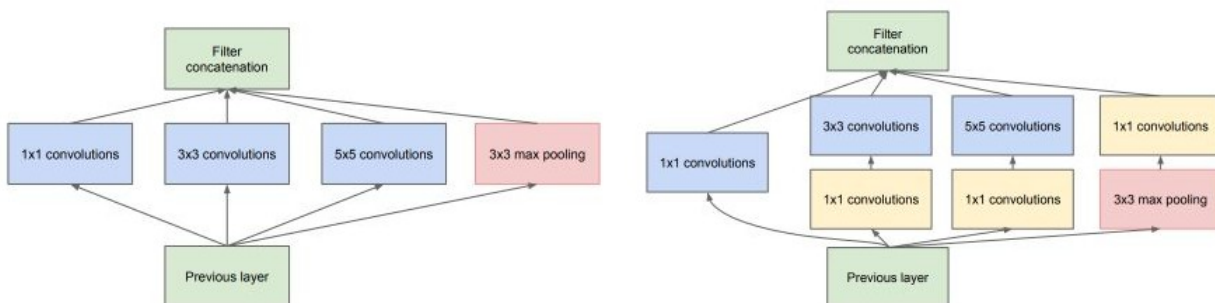


Figure 1.6: Inception module: naive version (left) and with dimensionality reduction along the channels dimension (right).



Figure 1.7: Full GoogLeNet architecture. “...” denote more Inception modules and auxiliary classifiers.

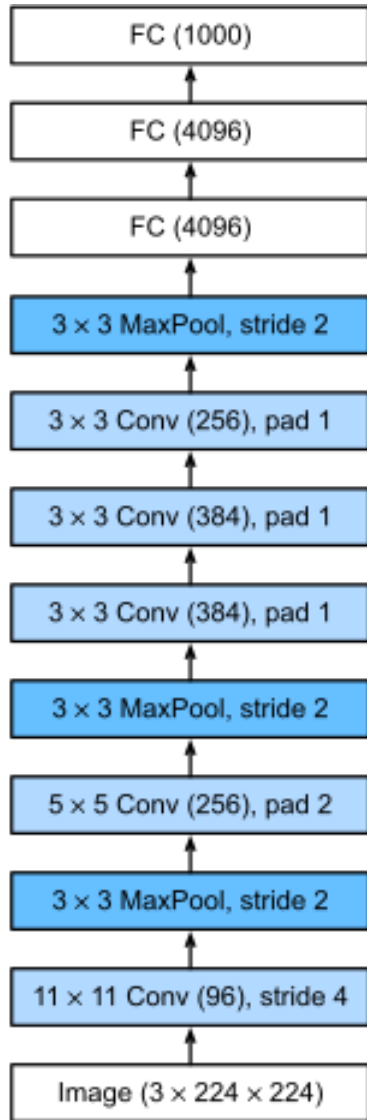


Figure 1.8: AlexNet.

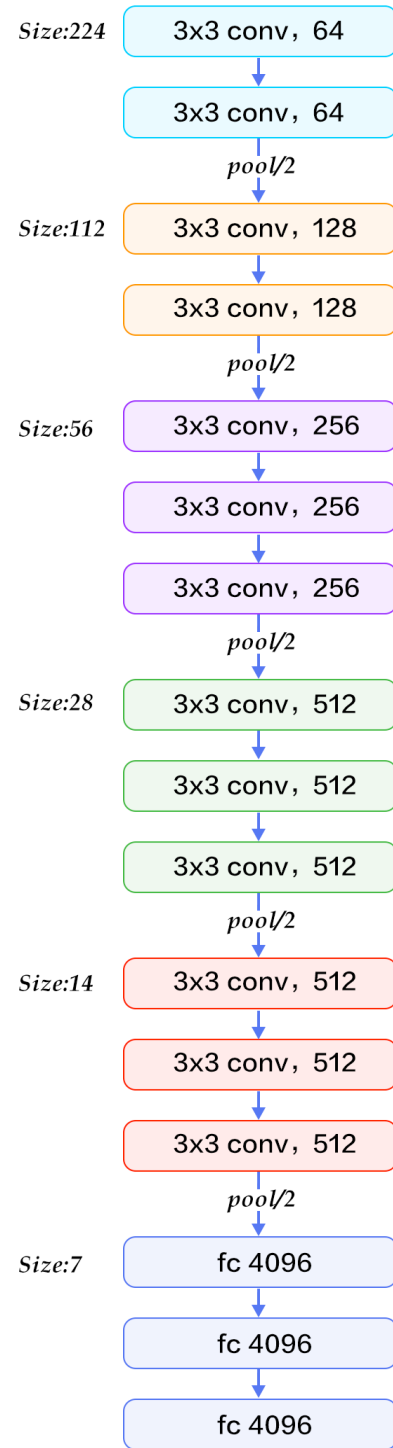


Figure 1.9: VGGNet.

1.2.5 ResNet (2015)

ResNet (Residual Network) fundamentally altered how deep architectures are optimized, solving the “degradation problem” where adding more layers to a sufficiently deep network paradoxically increased training error. Figure 1.10 shows the main idea.

Key innovations:

- **Residual Connections & Gradient Flow:** Instead of forcing layers to directly fit an unreferenced underlying mapping $\mathcal{H}(x)$, the network reformulates the target as a residual mapping $\mathcal{F}(x) = \mathcal{H}(x) - x$. The block output is thus given by $y = \mathcal{F}(x) + x$.

During backpropagation, applying the chain rule to compute the gradient of the loss $\mathcal{L}$ with respect to the block input x yields:

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial y} \frac{\partial y}{\partial x} = \frac{\partial \mathcal{L}}{\partial y} \left(\frac{\partial \mathcal{F}(x) + x}{\partial x} \right) = \frac{\partial \mathcal{L}}{\partial y} \left(\frac{\partial \mathcal{F}(x)}{\partial x} + 1 \right) = \frac{\partial \mathcal{L}}{\partial y} + \frac{\partial \mathcal{L}}{\partial y} \frac{\partial \mathcal{F}(x)}{\partial x}$$

The presence of the constant $+1$ guarantees that the gradient $\partial \mathcal{L} / \partial y$ flows back directly to x unimpeded. Even if the weight layers within $\mathcal{F}(x)$ suffer from vanishing gradients (i.e., $\partial \mathcal{F}(x) / \partial x \rightarrow 0$), the signal is preserved by the additive $+1$ term, ensuring robust gradient flow throughout the entire network.

- **Ease of Learning Identity Mappings:** If a newly added layer is unnecessary for performance, the network can easily drive the residual weights $\mathcal{F}(x)$ to zero, leaving an identity transformation ($y = x$) that preserves performance.
- **Extreme Depth:** Enabled the successful training of networks with 101, 152, and even 1000+ layers without suffering from gradient degradation, setting a new standard for feature extraction.

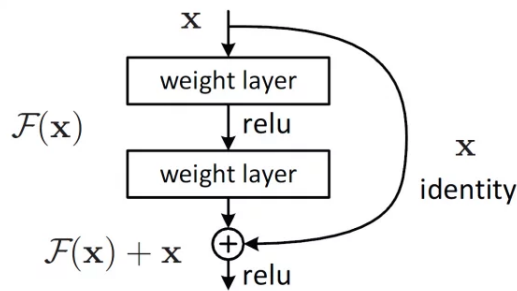


Figure 1.10: Residual module.

1.2.6 DenseNet (2017)

DenseNet (Densely Connected Convolutional Networks) pushed the concept of skip connections to its logical extreme, fundamentally shifting the focus from summing feature maps to concatenating them (Figure 1.11).

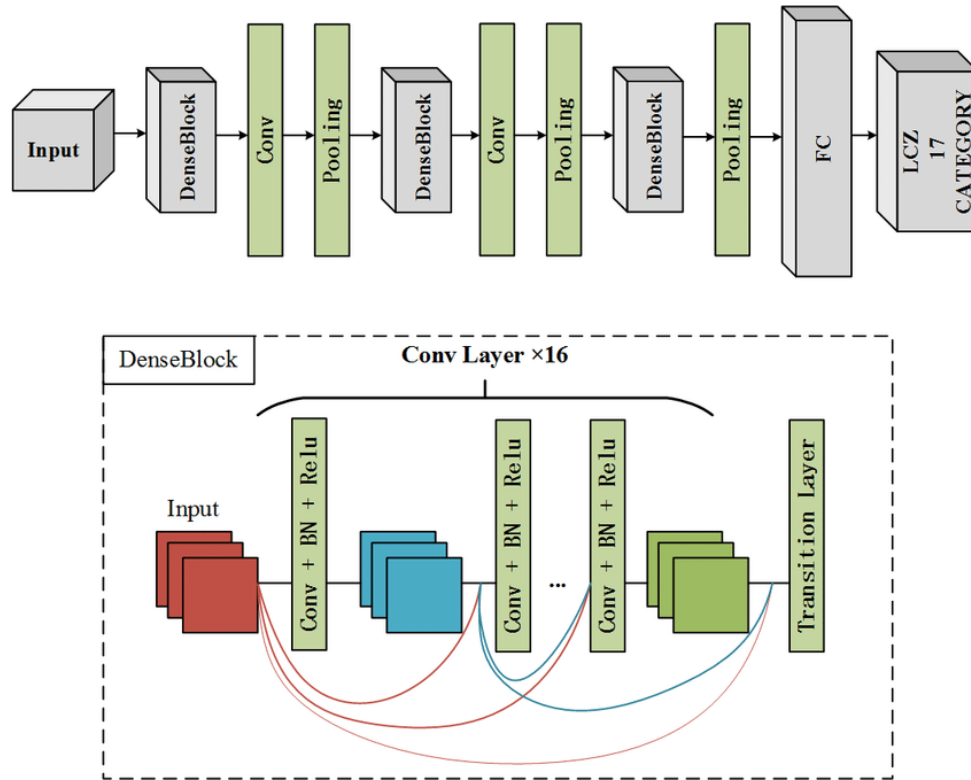


Figure 1.11: Example of DenseNet and a dense module.

Key innovations:

- **Dense Connectivity:** Within a dense block, each layer receives the concatenated feature maps of *all* preceding layers, and passes its own feature maps to all subsequent layers.
- **Feature Reuse:** Maximizes information flow. Deep layers can directly access low-level visual features extracted by the very first layers, creating a highly diverse feature space.
- **Parameter Efficiency:** Because features are continuously aggregated rather than re-learned, each layer only needs to output a very small number of new feature maps (controlled by a hyperparameter called the “growth rate”). This allows DenseNets to achieve state-of-the-art accuracy with a fraction of the parameters used by equivalent ResNets.

1.3 Transfer Learning

1.3.1 Transferring Knowledge

Transfer learning is a machine learning technique where a model developed for one task is reused as the starting point for a different but related task. Instead of training a model

from scratch, you take a pretrained model (usually trained on a large dataset) and fine-tune it or use it as a feature extractor for your specific problem.

This approach is especially useful when you have limited data for your target task or if training a large model from scratch would be too expensive. In deep learning, it is common in computer vision and natural language processing because pretrained models capture general patterns (like edges in images or grammar in text) that can be adapted to new tasks.

1.3.2 Fine-tuning

In practice, neural networks are seldom trained completely from scratch (i.e., with random initialization). Instead, one typically builds upon weights obtained from prior training. You can either use the pretrained model as a fixed feature extractor (appending a separate classifier) or you can replace the final layer with a new linear classifier (adjusting its size to the new number of target classes) and resume training. In this case you, can fine-tune all or just some of its layers.

Depending on the size of the new dataset and its similarity to the original training data, broadly four scenarios can be distinguished:

- **Large, similar dataset.** In this scenario, you can start by training only the last (task-specific) layer. If data and computer resources permit, you can gradually unfreeze previous layers as training stabilizes.
- **Large, dissimilar dataset.** Here, you can fine-tune every layer of the pretrained model, but if the new domain (e.g., medical imaging vs. ImageNet) truly diverges, the original weights may offer limited benefit. If compute and data resources permit, consider training from scratch on a more closely related dataset. This can sometimes converge faster than heavy fine-tuning of mismatched weights.
- **Small, similar dataset.** In this case, you can freeze the pretrained network and train only the last layer. Alternatively, you can use the frozen backbone as a feature extractor and train an external classifier.
- **Small, dissimilar dataset.** In this situation, the pretrained representations may not align with the new domain. A possible starting point is to freeze the backbone and extract features from intermediate layers to determine if the learned features adapt to the new data. As an alternative, you can consider other strategies such as few-shot learning.

	Similar dataset	Different dataset
A lot of data	Fine-tune some or all model layers	Either fine-tune all model layers or train from scratch!
Little data	Use linear classifier on final layer or use the backbone as feature extractor	(Bad scenario) Extract features from intermediate layers or try another model/strategy

Table 1.1: Starting point heuristic for fine-tuning.

1.4 Exercises

Exercise 1: Computer vision architectures

Create a table comparing all the famous architectures seen in this chapter. Include columns for the year of introduction, key innovations, and their primary structural characteristics (e.g., use of Inception modules, residual connections, or dense blocks).

Exercise 2: Fine-tuning

Load a pretrained model (e.g., ResNet-18) using PyTorch and fine-tune it on the MNIST dataset. Note that MNIST images are grayscale (1 channel), 28×28 pixels, and 10 classes, so you will need to adapt the input layer and the final classification head accordingly. Also, you can upscale the images to the original training size of the network to avoid downsampling very aggressively.

Exercise 3: Convolutional arithmetic

Given an input image tensor of shape $(C_{\text{in}}, H, W) = (3, 224, 224)$ and a convolutional layer with 64 filters of size 7×7 , a stride of $s = 2$, padding $p = 3$, and dilation $d = 1$. Use the standard output shape equations to calculate the exact shape of the resulting output tensor Y .

Exercise 4: Gradient Flow in ResNets

Explain mathematically how residual connections help mitigate optimization and vanishing-gradient difficulties. In your answer, reference the derivative $\partial \mathcal{L} / \partial x$ and explain the significance of the +1 term during backpropagation.

Exercise 5: Receptive field

Consider a convolutional neural network composed of five consecutive 3×3 convolutional layers. Assume a unit stride ($s = 1$) and no dilation ($d = 1$) for all layers.

1. Mathematical Derivation: Using the general recursive formula for receptive field expansion, calculate the final effective receptive field size RF_5 of a single unit in Layer 5 relative to the original input space. Show your step-by-step calculations for each layer $L \in \{1, 2, 3, 4, 5\}$.
2. Visual Representation: Draw a diagram illustrating the step-by-step expansion of the receptive field from Layer 5 back down to the input space. Clearly label the spatial dimension (in pixels) of the receptive field at each layer.

Exercise 6: Transfer Learning Strategy

You are tasked with developing a deep learning system to classify X-ray images of a rare bone disease, and you only have access to a dataset of 400 labeled images. If you are starting from a model pretrained on ImageNet (a dataset of everyday objects like dogs, cars, and trees), which transfer learning scenario from the Selection Guide applies? Describe your exact strategy for fine-tuning.

Exercise 7: Data Augmentation

Data augmentation acts as a powerful spatial and intensity regularizer. Write a PyTorch code snippet using the `torchvision.transforms` module to define a training pipeline that includes a random crop, a horizontal flip, and random color jittering. Visualize the original and modified images.

Exercise 8: Feature Extraction

Using PyTorch, load a pretrained `vgg16` model. Write the code to freeze all of its convolutional parameters (the feature extractor) so they do not update during back-propagation. Then, replace the final fully connected layer so that the network outputs predictions for 10 custom classes.